```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0, viewport-fit=cover">
  <title>Weather Gate v2.5</title>
  <link rel="stylesheet" href="uPlot.min.css">
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    :root {
      --bg: #f5f5f7;
      --card: #ffffff;
      --text: #1e1e1e;
      --text-secondary: #6c6c74;
      --temp: #ff6b35;
      --humidity: #2196f3;
      --pressure: #4caf50;
      --wind: #9c27b0;
      --rain: #00bcd4;
      --battery: #ff9800;
      --success: #4caf50;
      --error: #f44336;
      --warning: #ff9800;
      --radius: 24px;
      --shadow: 0 2px 8px rgba(0,0,0,0.05);
      --forecast: #ff6b35;
    }
    body.dark {
      --bg: #121212;
      --card: #1e1e1e;
      --text: #e3e3e3;
      --text-secondary: #9e9e9e;
    }
    body {
      font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
      background: var(--bg);
      padding: 12px;
      color: var(--text);
      transition: background 0.3s, color 0.3s;
    }
    .container { max-width: 550px; margin: 0 auto; display: flex; flex-direction: column; gap: 12px; }
    .cards-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
    .m-card { background: var(--card); border-radius: var(--radius); padding: 14px; box-shadow: var(--shadow); }
    .m-card.wide { grid-column: span 2; }
```

```
.card-header { display: flex; align-items: center; gap: 12px; margin-bottom: 8px; }
 .card-icon { width: 44px; height: 44px; border-radius: 50%; display: flex; align-items: center; justify-content: center; font-size: 22px; }
.card-info { flex: 1; }
 .card-label { font-size: 12px; font-weight: 600; color: var(--text-secondary); text-transform: uppercase; letter-spacing: 0.5px; }
.card-value { font-size: 28px; font-weight: 800; line-height: 1.2; }
.card-sub { font-size: 12px; font-weight: 500; color: var(--text-secondary); margin-top: 4px; }
 .status-row { display: flex; justify-content: space-between; margin-top: 12px; font-size: 13px; font-weight: 500; color: var(--text-secondary); }
.status-row b { color: var(--text); font-weight: 700; }
.badge { display: inline-block; padding: 4px 8px; border-radius: 12px; font-size: 11px; font-weight: 700; }
.badge-success { background: rgba(76,175,80,0.15); color: var(--success); }
.badge-error { background: rgba(244,67,54,0.15); color: var(--error); }
.badge-warning { background: rgba(255,152,0,0.15); color: var(--warning); }

/* Роза ветров 16 направлений */
.compass { position: relative; width: 140px; height: 140px; margin: 10px auto; }
.compass-bg {
  width: 140px;
  height: 140px;
  background: var(--bg);
  border-radius: 50%;
  position: relative;
  background-image: radial-gradient(circle at center, transparent 50%, rgba(0,0,0,0.05) 100%);
}
.compass-arrow {
  position: absolute;
  top: 50%;
  left: 50%;
  width: 4px;
  height: 55px;
  background: var(--wind);
  transform-origin: 50% 0%;
  transition: transform 0.5s ease;
  z-index: 2;
}
.compass-arrow::after {
  content: "▼";
  position: absolute;
  bottom: -10px;
  left: -6px;
  font-size: 16px;
  color: var(--wind);
```

```
    }
    .compass-label {
      text-align: center;
      margin-top: 8px;
      font-size: 14px;
      font-weight: bold;
      color: var(--wind);
    }
    /* Метки направлений на розе */
    .compass-mark {
      position: absolute;
      font-size: 10px;
      font-weight: 600;
      color: var(--text-secondary);
      transform: translate(-50%, -50%);
    }

     .forecast-card { background: var(--card); border-radius: var(--radius); padding: 14px; box-shadow: var(--shadow); text-align: center; grid-column: span 2; }
    .forecast-text { font-size: 22px; font-weight: 800; color: var(--forecast); margin: 8px 0; }
    .forecast-icon { font-size: 40px; margin-bottom: 6px; }

    .freq-panel { background: var(--card); border-radius: var(--radius); padding: 14px; margin-top: 8px; }
    .freq-value { font-size: 32px; font-weight: 800; text-align: center; color: var(--temp); font-family: monospace; }

    .settings-panel { background: var(--card); border-radius: var(--radius); margin-top: 8px; }
      .settings-header { padding: 16px; display: flex; justify-content: space-between; align-items: center; cursor: pointer; font-weight: 700; }
    .settings-content { max-height: 0; overflow: hidden; transition: max-height 0.3s ease-out; padding: 0 16px; }
    .settings-content.open { max-height: 2000px; padding: 0 16px 16px 16px; }
    .settings-row { display: flex; gap: 12px; margin-bottom: 12px; flex-wrap: wrap; }
    .settings-row > div { flex: 1; min-width: 120px; }
    label { font-size: 11px; font-weight: 600; color: var(--text-secondary); display: block; margin-bottom: 4px; }
     input { width: 100%; padding: 10px 12px; border-radius: 12px; border: none; background: var(--bg); color: var(--text); font-size: 14px; font-weight: 500; box-sizing: border-box; }
     button { width: 100%; padding: 12px; border-radius: 12px; border: none; background: var(--temp); color: white; font-weight: 700; cursor: pointer; transition: transform 0.1s; }
    button:active { transform: scale(0.98); }
    .btn-grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; margin-top: 12px; }
      .theme-toggle { background: var(--bg); border-radius: 20px; padding: 12px; font-size: 12px; cursor: pointer; width: auto; color: var(--text); }
    .update-panel { background: var(--card); border-radius: var(--radius); padding: 12px; margin-top: 8px; display: none; }
     .update-status { font-size: 13px; text-align: center; padding: 8px; border-radius: 12px; background: var(--bg); margin-bottom: 8px; }
```

```
      .power-progress { margin-top: 12px; }
          .power-labels { display: flex; justify-content: space-between; font-size: 10px; font-weight: 500; color: var(--text-secondary); margin-bottom: 4px; }
      .progress-bar { width: 100%; height: 6px; background: rgba(0,0,0,0.1); border-radius: 3px; overflow: hidden; }
      .progress-fill { height: 100%; width: 0%; transition: width 0.3s; border-radius: 3px; }

      .chart-container { margin-top: 12px; width: 100%; height: 140px; }
        #spectrumCanvas { cursor: crosshair; transition: background 0.2s; border-radius: 16px; width: 100%; height: 180px; background: var(--bg); }
      #spectrumCanvas:hover { background: rgba(33,150,243,0.05) !important; }

      @media (max-width: 480px) {
        .status-row { flex-direction: column; gap: 6px; }
        button { padding: 14px; font-size: 14px; }
        .card-value { font-size: 22px; }
        .btn-grid { grid-template-columns: repeat(2, 1fr); }
        .compass { width: 110px; height: 110px; }
        .compass-arrow { height: 42px; }
        .compass-mark { font-size: 8px; }
      }
    </style>
</head>
<body>
<div class="container">
    <!-- Первый ряд: Температура и Влажность -->
    <div class="cards-grid">
      <!-- Температура -->
      <div class="m-card">
        <div class="card-header">
          <div class="card-icon" style="background:rgba(255,107,53,0.1); color: var(--temp);">🌡️</div>
          <div class="card-info">
            <div class="card-label">Температура</div>
            <div class="card-value"><span id="temperature">--</span> °C</div>
          </div>
        </div>
        <div class="status-row">
          <span>Мин: <b><span id="temp_min">--</span> °C</b></span>
          <span>Макс: <b><span id="temp_max">--</span> °C</b></span>
        </div>
        <div class="power-progress">
            <div class="power-labels"><span>-30°C</span><span>0°C</span><span>+30°C</span><span>+50°C</span></div>
          <div class="progress-bar"><div id="temp-fill" class="progress-fill" style="background: var(--temp);"></div></div>
```

```
      </div>
    </div>

    <!-- Влажность -->
    <div class="m-card">
      <div class="card-header">
        <div class="card-icon" style="background:rgba(33,150,243,0.1); color: var(--humidity);">💧</div>
        <div class="card-info">
          <div class="card-label">Влажность</div>
          <div class="card-value"><span id="humidity">--</span> %</div>
        </div>
      </div>
      <div class="power-progress">
                        <div class="power-labels"><span>0%</span><span>25%</span><span>50%</span><span>75%</span><span>100%</span></div>
                            <div class="progress-bar"><div id="hum-fill" class="progress-fill" style="background: var(--humidity);"></div></div>
      </div>
    </div>
  </div>

  <!-- Второй ряд: Давление и Осадки -->
  <div class="cards-grid">
    <!-- Давление -->
    <div class="m-card">
      <div class="card-header">
        <div class="card-icon" style="background:rgba(76,175,80,0.1); color: var(--pressure);">📊</div>
        <div class="card-info">
          <div class="card-label">Давление</div>
          <div class="card-value"><span id="pressure">--</span> hPa</div>
        </div>
      </div>
      <div class="status-row">
        <span>Мин: <b><span id="pressure_min">--</span></b></span>
        <span>Макс: <b><span id="pressure_max">--</span></b></span>
      </div>
      <div class="power-progress">
        <div class="power-labels"><span>970</span><span>1013</span><span>1050</span></div>
                        <div class="progress-bar"><div id="pressure-fill" class="progress-fill" style="background: var(--pressure);"></div></div>
      </div>
    </div>

    <!-- Осадки -->
```

```
      <div class="m-card">
        <div class="card-header">
          <div class="card-icon" style="background:rgba(0,188,212,0.1); color: var(--rain);">☔</div>
          <div class="card-info">
            <div class="card-label">Осадки</div>
            <div class="card-value"><span id="rain">--</span> mm</div>
          </div>
        </div>
        <div class="status-row">
          <span>Макс: <b><span id="rain_max">--</span> mm</b></span>
        </div>
        <div class="power-progress">
                <div  class="power-labels"><span>0</span><span>10</span><span>25</span><span>50</span><span>100+
mm</span></div>
          <div class="progress-bar"><div id="rain-fill" class="progress-fill" style="background: var(--rain);"></div></div>
        </div>
      </div>
    </div>

    <!-- Третий ряд: Ветер (скорость) и Направление (роза ветров) -->
    <div class="cards-grid">
      <!-- Скорость ветра -->
      <div class="m-card">
        <div class="card-header">
          <div class="card-icon" style="background:rgba(156,39,176,0.1); color: var(--wind);">💨</div>
          <div class="card-info">
            <div class="card-label">Ветер</div>
            <div class="card-value"><span id="wind_speed">--</span> м/с</div>
            <div class="card-sub">Порывы: <span id="wind_gust">--</span> м/с</div>
          </div>
        </div>
        <div class="status-row">
          <span>Макс: <b><span id="wind_max">--</span> м/с</b></span>
        </div>
        <div class="power-progress">
                  <div  class="power-labels"><span>0</span><span>10</span><span>20</span><span>30</span><span>40
м/с</span></div>
          <div class="progress-bar"><div id="wind-fill" class="progress-fill" style="background: var(--wind);"></div></div>
        </div>
      </div>

      <!-- Направление ветра (роза ветров 16 направлений) -->
      <div class="m-card">
        <div class="card-header">
```

```
        <div class="card-icon" style="background:rgba(156,39,176,0.1); color: var(--wind);">🧭</div>
        <div class="card-info">
          <div class="card-label">Направление</div>
          <div class="card-value" id="wind_dir_deg" style="font-size: 20px;">--°</div>
          <div class="card-sub" id="wind_dir_txt">--</div>
        </div>
      </div>
      <div class="compass" id="compass">
        <div class="compass-bg" id="compassBg"></div>
        <div id="windArrow" class="compass-arrow"></div>
        <div id="windDirLabel" class="compass-label">--</div>
      </div>
    </div>
</div>

<!-- Прогноз Zambretti (широкий) -->
<div class="forecast-card">
    <div class="card-header">
      <div class="card-icon" style="background:rgba(255,107,53,0.1); color: var(--forecast);">🔮</div>
      <div class="card-info">
        <div class="card-label">Прогноз (Zambretti)</div>
        <div class="forecast-text" id="forecast_text">--</div>
        <div class="card-sub">на ближайшие 12-24 часа</div>
      </div>
    </div>
    <div id="forecast_icon" class="forecast-icon">⏳</div>
    <div class="status-row" style="justify-content: center;">
      <span>Тенденция: <b><span id="pressure_trend">--</span></b></span>
    </div>
</div>

<!-- График температуры -->
<div class="m-card wide">
    <div class="card-header">
      <div class="card-icon" style="background:rgba(255,107,53,0.1); color: var(--temp);">📈</div>
      <div class="card-info">
        <div class="card-label">Температура</div>
        <div class="card-sub">последние 24 часа (5-минутные интервалы)</div>
      </div>
    </div>
    <div id="temp-chart" class="chart-container"></div>
</div>

<!-- График давления -->
```

```
  <div class="m-card wide">
    <div class="card-header">
      <div class="card-icon" style="background:rgba(76,175,80,0.1); color: var(--pressure);">📉</div>
      <div class="card-info">
        <div class="card-label">Давление</div>
        <div class="card-sub">последние 24 часа (5-минутные интервалы)</div>
      </div>
    </div>
    <div id="pressure-chart" class="chart-container"></div>
  </div>

  <!-- Спектроанализатор -->
  <div class="m-card wide">
    <div class="card-header">
      <div class="card-icon" style="background:rgba(33,150,243,0.1); color: #2196F3;">📡</div>
      <div class="card-info">
        <div class="card-label">АЧХ радиоканала</div>
        <div class="card-sub">диапазон 914.80 - 915.20 MHz</div>
      </div>
    </div>
    <canvas id="spectrumCanvas" width="600" height="180"></canvas>
     <div style="display: flex; justify-content: space-between; align-items: center; margin-top: 10px; flex-wrap: wrap; gap: 8px;">
      <div style="display: flex; gap: 16px;">
        <span style="font-size: 13px; font-weight: 600; font-family: monospace;" id="current_freq">915.04</span>
        <span style="font-size: 13px;">MHz</span>
        <span style="font-size: 13px;">📦 <b id="radio_success_header">0</b></span>
        <span style="font-size: 13px;">⚠️ <b id="radio_errors_header">0</b></span>
        <span style="font-size: 13px;">📶 <span id="scan_rssi">--</span> dBm</span>
      </div>
          <button id="startScanBtn" onclick="startFullScan()" style="width: auto; padding: 8px 20px; background: var(--success);">🔍 Найти лучшую частоту</button>
    </div>
    <div class="status-row" style="margin-top: 8px; font-size: 11px;">
      <span>🔵 Синяя линия — уровень сигнала</span>
      <span>🔴 Пунктир — порог шума</span>
      <span>🟠 Маркер — текущая частота</span>
      <span>💡 Кликните по графику — переключиться</span>
    </div>
  </div>

  <!-- Система -->
  <div class="cards-grid">
    <div class="m-card">
```

```
      <div class="card-header">
        <div class="card-icon" style="background:rgba(76,175,80,0.1); color: var(--success);">⚙️</div>
        <div class="card-info">
          <div class="card-label">Система</div>
          <div class="card-value"><span id="version">2.5</span></div>
        </div>
      </div>
      <div class="status-row">
        <span>Питание: <b><span id="power_source">AC</span></b></span>
        <span>MQTT: <b><span id="mqtt_status">--</span></b></span>
      </div>
      <div class="status-row">
        <span>Uptime: <span id="uptime">0</span></span>
        <span>Радио: <b><span id="radio_status">OK</span></b></span>
        <span>ID датчика: <b><span id="sensor_id">--</span></b></span>
      </div>
      <div class="status-row">
        <span>Батарея: <b><span id="outdoor_battery">--</span></b></span>
        <span><span id="datetime"></span></span>
      </div>
    </div>
  </div>

  <!-- Кнопки управления -->
  <div class="m-card wide">
    <div class="btn-grid">
      <button class="secondary" onclick="syncTime()">🕐 Синхр. время</button>
      <button onclick="resetRadioErrors()" class="warning">📡 Сброс ошибок</button>
      <button onclick="toggleTheme()" class="theme-toggle">🌙 Тёмная тема</button>
    </div>
  </div>

  <!-- Настройки -->
  <div class="settings-panel">
    <div class="settings-header" onclick="toggleSettings()">
      <span>⚙️ НАСТРОЙКИ</span>
      <span>▼</span>
    </div>
    <div id="settingsContent" class="settings-content">
      <div class="settings-row">
        <div><label>Hostname</label><input type="text" id="hostname" placeholder="weather_gate"></div>
        <div><label>MQTT IP</label><input type="text" id="mq_ip" placeholder="192.168.1.100"></div>
      </div>
      <div class="settings-row">
```

```
          <div><label>Логин MQTT</label><input type="text" id="mq_u"></div>
          <div><label>Пароль MQTT</label><input type="password" id="mq_p"></div>
        </div>
        <div class="settings-row">
                                  <div><label>MQTT топик погоды</label><input type="text" id="mqtt_topic_weather"
placeholder="weather_gate/data"></div>
          <div><label>Коэфф. АКБ</label><input type="number" id="bat_k" step="0.01" value="2.10"></div>
        </div>
        <div class="btn-group" style="margin-top: 12px;">
          <button onclick="save()">💾 Сохранить</button>
        </div>
      </div>
    </div>
</div>

<script src="uPlot.iife.min.js"></script>
<script>
    let tempChart, pressureChart;
    let settingsOpen = false;
    let verboseMode = false;
    let maxTemp = 50, minTemp = -30;
    let maxPress = 1050, minPress = 970;
    let maxRain = 100;
    let scanActive = false;
    let scanResults = new Array(21).fill(-120);
    let scanCurrentStep = 0;
    let scanLiveRssi = -120;
    let scanInterval = null;

    // 16 направлений ветра
    const windDirections16 = [
      "С", "ССВ", "СВ", "ВСВ", "В", "ВЮВ", "ЮВ", "ЮЮВ",
      "Ю", "ЮЮЗ", "ЮЗ", "ЗЮЗ", "З", "ЗСЗ", "СЗ", "ССЗ"
    ];

    function getWindDirection16(deg) {
      const idx = Math.round(deg / 22.5) % 16;
      return windDirections16[idx];
    }

    // Отрисовка розы ветров (16 направлений)
    function drawCompassRose() {
      const container = document.getElementById('compassBg');
      if (!container) return;
```

```
container.innerHTML = '';
const size = 140;
const centerX = size / 2;
const centerY = size / 2;
const radius = size / 2 - 5;

for (let i = 0; i < 16; i++) {
    const angle = (i * 22.5 - 90) * Math.PI / 180;
    const x1 = centerX + radius * 0.85 * Math.cos(angle);
    const y1 = centerY + radius * 0.85 * Math.sin(angle);
    const x2 = centerX + radius * Math.cos(angle);
    const y2 = centerY + radius * Math.sin(angle);

    const isCardinal = (i % 4 === 0);
    const isHalf = (i % 2 === 0);

    const line = document.createElement('div');
    line.style.position = 'absolute';
    line.style.left = x1 + 'px';
    line.style.top = y1 + 'px';
    line.style.width = Math.hypot(x2 - x1, y2 - y1) + 'px';
    line.style.height = isCardinal ? '2px' : '1px';
    line.style.backgroundColor = isCardinal ? 'var(--wind)' : 'var(--text-secondary)';
    line.style.transformOrigin = '0 0';
    line.style.transform = `rotate(${i * 22.5}deg)`;
    line.style.opacity = isHalf ? 0.6 : 0.3;
    container.appendChild(line);
}

// Буквы для 4 основных направлений
const labels = [
    { dir: 'С', angle: -90, offset: 12 },
    { dir: 'В', angle: 0, offset: 12 },
    { dir: 'Ю', angle: 90, offset: 12 },
    { dir: 'З', angle: 180, offset: 12 }
];
labels.forEach(label => {
    const rad = label.angle * Math.PI / 180;
    const x = centerX + (radius + label.offset) * Math.cos(rad);
    const y = centerY + (radius + label.offset) * Math.sin(rad);
    const div = document.createElement('div');
    div.className = 'compass-mark';
    div.style.left = x + 'px';
    div.style.top = y + 'px';
```

```
        div.style.transform = 'translate(-50%, -50%)';
        div.style.fontWeight = 'bold';
        div.style.color = 'var(--wind)';
        div.innerText = label.dir;
        container.appendChild(div);
      });
    }

    function initTheme() {
      const saved = localStorage.getItem('darkMode');
      if (saved === 'true') document.body.classList.add('dark');
      else if (saved === 'false') document.body.classList.remove('dark');
      else if (window.matchMedia('(prefers-color-scheme: dark)').matches) document.body.classList.add('dark');
      setTimeout(() => drawCompassRose(), 100);
    }

    function toggleTheme() {
      document.body.classList.toggle('dark');
      localStorage.setItem('darkMode', document.body.classList.contains('dark'));
      setTimeout(() => drawCompassRose(), 100);
      if (tempChart) tempChart.destroy();
      if (pressureChart) pressureChart.destroy();
      initCharts();
      loadHistory();
      drawSpectrum();
    }

    function initCharts() {
      const commonOpts = {
        width: document.getElementById('temp-chart')?.offsetWidth || 400,
        height: 140,
        series: [{}],
        axes: [{ show: false }, { show: true, grid: { show: false } }],
        cursor: { drag: { setScale: false } }
      };
      if (document.getElementById('temp-chart')) {
        if (tempChart) tempChart.destroy();
        tempChart = new uPlot({...commonOpts, series: [{ stroke: "#ff6b35", fill: "rgba(255,107,53,0.1)", spanGaps: false }]},
[[], []], document.getElementById('temp-chart'));
      }
      if (document.getElementById('pressure-chart')) {
        if (pressureChart) pressureChart.destroy();
            pressureChart = new uPlot({...commonOpts, series: [{ stroke: "#4caf50", fill: "rgba(76,175,80,0.1)", spanGaps:
false }]}, [[], []], document.getElementById('pressure-chart'));
```

```
    }
}

function prepareData(values, stepSeconds) {
    const now = Math.floor(Date.now() / 1000);
    const timestamps = [];
    for (let i = values.length - 1; i >= 0; i--) {
        timestamps.push(now - (values.length - 1 - i) * stepSeconds);
    }
    return [timestamps, values];
}

function formatUptime(seconds) {
    const days = Math.floor(seconds / 86400);
    const hours = Math.floor((seconds % 86400) / 3600);
    const mins = Math.floor((seconds % 3600) / 60);
    if (days > 0) return `${days}д ${hours}ч`;
    if (hours > 0) return `${hours}ч ${mins}м`;
    return `${mins}м`;
}

function updateProgress(value, minVal, maxVal, fillId) {
    const percent = Math.min(Math.max((value - minVal) / (maxVal - minVal) * 100, 0), 100);
    const fill = document.getElementById(fillId);
    if (fill) fill.style.width = percent + '%';
}

function getForecastIcon(txt) {
    if (!txt) return "⏳";
    const t = txt.toLowerCase();
    if (t.includes('солнечн') || t.includes('ясн') || t.includes('отличная')) return "☀";
    if (t.includes('облачн') || t.includes('переменн')) return "⛅";
    if (t.includes('дожд') || t.includes('ливн')) return "🌧";
    if (t.includes('гроз') || t.includes('storm')) return "⛈";
    if (t.includes('туман')) return "🌫";
    return "🔮";
}

async function load() {
    try {
        const response = await fetch('/api/data');
        const d = await response.json();

        // Температура
```

```
        const temp = d.temperature || 0;
        document.getElementById('temperature').innerText = temp.toFixed(1);
        document.getElementById('temp_min').innerText = d.temp_min?.toFixed(1) || '--';
        document.getElementById('temp_max').innerText = d.temp_max?.toFixed(1) || '--';
        updateProgress(temp, minTemp, maxTemp, 'temp-fill');

        // Влажность
        const hum = d.humidity || 0;
        document.getElementById('humidity').innerText = hum.toFixed(0);
        updateProgress(hum, 0, 100, 'hum-fill');

        // Давление
        const press = d.pressure || 1013;
        document.getElementById('pressure').innerText = press.toFixed(0);
        document.getElementById('pressure_min').innerText = d.pressure_min?.toFixed(0) || '--';
        document.getElementById('pressure_max').innerText = d.pressure_max?.toFixed(0) || '--';
        updateProgress(press, minPress, maxPress, 'pressure-fill');

        // Ветер
        const wind = d.wind_speed || 0;
        document.getElementById('wind_speed').innerText = wind.toFixed(1);
        document.getElementById('wind_gust').innerText = d.wind_gust?.toFixed(1) || '--';
        document.getElementById('wind_max').innerText = d.wind_max?.toFixed(1) || '--';
        updateProgress(wind, 0, 40, 'wind-fill');

        // Направление ветра (роза)
        const windDir = d.wind_direction || 0;
        document.getElementById('wind_dir_deg').innerText = windDir + '°';
        document.getElementById('wind_dir_txt').innerText = getWindDirection16(windDir);
        const arrow = document.getElementById('windArrow');
        if (arrow) arrow.style.transform = `rotate(${windDir}deg) translateX(-50%)`;
        document.getElementById('windDirLabel').innerHTML = `${windDir}°<br>${getWindDirection16(windDir)}`;

        // Осадки
        const rain = d.rain || 0;
        document.getElementById('rain').innerText = rain.toFixed(1);
        document.getElementById('rain_max').innerText = d.rain_max?.toFixed(1) || '--';
        updateProgress(rain, 0, maxRain, 'rain-fill');

        // Система
        document.getElementById('power_source').innerText = d.power_source === 'AC' ? 'Сеть' : 'АКБ';
            document.getElementById('mqtt_status').innerHTML = d.mqtt_status === 'OK' ? '<span class="badge badge-
success">OK</span>' : '<span class="badge badge-error">Offline</span>';
        document.getElementById('version').innerText = d.version || '--';
```

```
        document.getElementById('uptime').innerText = formatUptime(d.uptime || 0);
        document.getElementById('radio_success_header').innerText = d.radio_success || 0;
        document.getElementById('radio_errors_header').innerText = d.radio_errors || 0;
        document.getElementById('datetime').innerText = d.datetime || '--';
        document.getElementById('sensor_id').innerText = d.sensor_id || '--';
            document.getElementById('outdoor_battery').innerHTML = d.outdoor_battery === 'OK' ? '<span class="badge badge-success">OK</span>' : '<span class="badge badge-error">LOW</span>';
        document.getElementById('current_freq').innerText = d.current_freq?.toFixed(2) || '915.00';

        // Радио статус
        const radioErrors = d.radio_errors || 0;
        const radioSuccess = d.radio_success || 0;
        const rs = document.getElementById('radio_status');
        if (rs) {
          if (radioErrors === 0 && radioSuccess === 0) {
            rs.innerText = 'НЕТ ДАННЫХ';
            rs.className = 'badge badge-warning';
          } else if (radioErrors > 0 && radioSuccess === 0) {
            rs.innerText = 'ОШИБКИ';
            rs.className = 'badge badge-error';
          } else {
            rs.innerText = 'OK';
            rs.className = 'badge badge-success';
          }
        }

        // Прогноз
        if (d.forecast) {
          document.getElementById('forecast_text').innerHTML = d.forecast;
          document.getElementById('forecast_icon').innerHTML = getForecastIcon(d.forecast);
        }
        if (d.pressure_trend) document.getElementById('pressure_trend').innerHTML = d.pressure_trend;

        if (!scanActive && d.rssi) document.getElementById('scan_rssi').innerText = d.rssi;

        // Настройки
        if (document.getElementById('hostname')) {
          document.getElementById('hostname').value = d.hostname || 'weather_gate';
          document.getElementById('mq_ip').value = d.mqtt_ip || '';
          document.getElementById('mq_u').value = d.mqtt_user || '';
          document.getElementById('mq_p').value = d.mqtt_password || '';
          document.getElementById('bat_k').value = d.battery_coeff || 2.1;
          document.getElementById('mqtt_topic_weather').value = d.mqtt_topic_weather || 'weather_gate/data';
        }
```

```
    } catch(e) { console.error(e); }
}

async function loadHistory() {
    try {
        const response = await fetch('/api/history');
        const d = await response.json();
        if (d.temp_history && tempChart) tempChart.setData(prepareData(d.temp_history, 300));
        if (d.pressure_history && pressureChart) pressureChart.setData(prepareData(d.pressure_history, 300));
    } catch(e) { console.error(e); }
}

async function syncTime() {
    if (!confirm('Синхронизировать время?')) return;
    const now = Math.floor(Date.now() / 1000);
    await fetch('/api/sync_time', { method: 'POST', body: 't=' + now });
    alert('Время синхронизировано');
    load();
}

async function resetRadioErrors() {
    if (confirm('Сбросить счётчики ошибок?')) {
        await fetch('/api/reset_errors', { method: 'POST' });
        alert('Счётчики сброшены');
        load();
    }
}

async function save() {
    if (!confirm('Сохранить настройки и перезагрузить?')) return;
    const obj = {
        hostname: document.getElementById('hostname').value,
        mq_ip: document.getElementById('mq_ip').value,
        mq_u: document.getElementById('mq_u').value,
        mq_p: document.getElementById('mq_p').value,
        bat_k: document.getElementById('bat_k').value,
        mqtt_topic_weather: document.getElementById('mqtt_topic_weather').value
    };
    await fetch('/api/config', { method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(obj) });
    location.reload();
}

function toggleSettings() {
    const content = document.getElementById('settingsContent');
```

```
    if (settingsOpen) content.classList.remove('open');
    else content.classList.add('open');
    settingsOpen = !settingsOpen;
  }

  // Сканер
  async function startFullScan() {
    if (scanActive) { alert("Сканирование уже выполняется!"); return; }
        if (!confirm("Запустить автоматический поиск оптимальной частоты?\nЭто займёт около 23 минут. Приём данных временно приостановится.")) return;
    document.getElementById('startScanBtn').disabled = true;
    document.getElementById('startScanBtn').innerHTML = '⏳ Сканирование...';
    scanResults = new Array(21).fill(-120);
    scanCurrentStep = 0;
    scanActive = true;
    try {
      await fetch('/api/start_scan');
      if (scanInterval) clearInterval(scanInterval);
      scanInterval = setInterval(pollScanData, 2000);
                        } catch(e) { console.error(e); document.getElementById('startScanBtn').disabled = false; document.getElementById('startScanBtn').innerHTML = '🔍 Найти лучшую частоту'; scanActive = false; }
  }

  async function pollScanData() {
    try {
      const response = await fetch('/api/scan_data');
      const data = await response.json();
      scanResults = data.rssi;
      scanCurrentStep = data.step;
      scanActive = data.active;
      scanLiveRssi = data.instant || data.rssi_current;
      drawSpectrum();
      if (data.active) {
        const progress = Math.round((data.step / 21) * 100);
        document.getElementById('startScanBtn').innerHTML = `Сканирование: ${progress}%`;
        document.getElementById('scan_rssi').innerText = scanLiveRssi?.toFixed(0) || '--';
      } else {
        if (scanInterval) clearInterval(scanInterval);
        scanInterval = null;
        document.getElementById('startScanBtn').disabled = false;
        document.getElementById('startScanBtn').innerHTML = '🔍 Найти лучшую частоту';
        const dataResp = await fetch('/api/data');
        const dd = await dataResp.json();
        if (dd.rssi) document.getElementById('scan_rssi').innerText = dd.rssi;
```

```
        }
    } catch(e) { console.error(e); }
}

function drawSpectrum() {
    const canvas = document.getElementById('spectrumCanvas');
    if (!canvas) return;
    const w = canvas.offsetWidth;
    const h = canvas.offsetHeight;
    canvas.width = w;
    canvas.height = h;
    const ctx = canvas.getContext('2d');
    ctx.clearRect(0, 0, w, h);
    const bgColor = getComputedStyle(document.body).getPropertyValue('--bg').trim();
    ctx.fillStyle = bgColor;
    ctx.fillRect(0, 0, w, h);

    const noiseFloor = -108;
    const noiseY = h - ((noiseFloor + 120) * (h / 90));
    ctx.setLineDash([5, 5]);
    ctx.strokeStyle = '#f44336';
    ctx.lineWidth = 1;
    ctx.beginPath();
    ctx.moveTo(0, noiseY);
    ctx.lineTo(w, noiseY);
    ctx.stroke();
    ctx.setLineDash([]);
    ctx.fillStyle = '#f44336';
    ctx.font = '9px sans-serif';
    ctx.fillText('ШУМ', 5, noiseY - 3);

    const stepX = w / (scanResults.length - 1);
    ctx.beginPath();
    ctx.lineWidth = 2.5;
    ctx.lineJoin = 'round';
    ctx.strokeStyle = scanActive ? '#2196F3' : '#4CAF50';
    let firstPoint = true;
    for (let i = 0; i < scanResults.length; i++) {
        let val = scanResults[i];
        if (val === null || val === -120) continue;
        let y = h - ((Math.max(-120, Math.min(-30, val)) + 120) * (h / 90));
        let x = i * stepX;
        if (firstPoint) {
            ctx.moveTo(x, y);
```

```
            firstPoint = false;
        } else {
            ctx.lineTo(x, y);
        }
    }
    ctx.stroke();

    ctx.fillStyle = '#2196F3';
    for (let i = 0; i < scanResults.length; i++) {
        let val = scanResults[i];
        if (val === null || val === -120) continue;
        let y = h - ((Math.max(-120, Math.min(-30, val)) + 120) * (h / 90));
        let x = i * stepX;
        ctx.beginPath();
        ctx.arc(x, y, 2, 0, Math.PI * 2);
        ctx.fill();
    }

    ctx.fillStyle = '#9e9e9e';
    ctx.font = '9px sans-serif';
    ctx.textAlign = 'center';
    const marks = [
        { freq: 914.80, label: '914.8' },
        { freq: 915.00, label: '915.0' },
        { freq: 915.20, label: '915.2' }
    ];
    marks.forEach(mark => {
        const x = ((mark.freq - 914.80) / 0.4) * w;
        if (x >= 0 && x <= w) {
            ctx.beginPath();
            ctx.moveTo(x, h);
            ctx.lineTo(x, h - 6);
            ctx.stroke();
            ctx.fillText(mark.label + ' MHz', x, h - 10);
        }
    });

    const curFreq = parseFloat(document.getElementById('current_freq').innerText) || 915.00;
    const markerX = ((curFreq - 914.80) / 0.4) * w;
    if (markerX >= 0 && markerX <= w) {
        ctx.setLineDash([5, 5]);
        ctx.strokeStyle = '#FF9800';
        ctx.lineWidth = 2;
        ctx.beginPath();
```

```
        ctx.moveTo(markerX, 0);
        ctx.lineTo(markerX, h);
        ctx.stroke();
        ctx.setLineDash([]);
        ctx.fillStyle = '#FF9800';
        ctx.beginPath();
        ctx.arc(markerX, 8, 5, 0, Math.PI * 2);
        ctx.fill();
        ctx.fillStyle = '#fff';
        ctx.font = 'bold 8px sans-serif';
        ctx.fillText(curFreq.toFixed(2), markerX, 12);
    }

    if (scanActive && scanLiveRssi && scanLiveRssi > -120) {
        const liveX = scanCurrentStep * stepX;
        const liveY = h - ((Math.max(-120, Math.min(-30, scanLiveRssi)) + 120) * (h / 90));
        ctx.beginPath();
        ctx.arc(liveX, liveY, 6, 0, Math.PI * 2);
        ctx.fillStyle = '#FF9800';
        ctx.fill();
        ctx.beginPath();
        ctx.arc(liveX, liveY, 3, 0, Math.PI * 2);
        ctx.fillStyle = '#fff';
        ctx.fill();
        ctx.fillStyle = '#FF9800';
        ctx.font = 'bold 9px sans-serif';
        ctx.fillText(scanLiveRssi.toFixed(0) + ' dBm', liveX + 8, liveY - 5);
    }
}

function setupSpectrumClick() {
    const canvas = document.getElementById('spectrumCanvas');
    if (!canvas) return;
    canvas.onclick = async function(e) {
        if (scanActive) { alert("Сканирование активно, подождите его завершения"); return; }
        const rect = canvas.getBoundingClientRect();
        const x = e.clientX - rect.left;
        const w = canvas.offsetWidth;
        const clickedFreq = (914.80 + (x / w) * 0.4).toFixed(3);
        if (confirm(`Настроить приёмник на частоту ${clickedFreq} MHz?`)) {
            await fetch('/api/set_freq', { method: 'POST', body: 'f=' + clickedFreq });
            document.getElementById('current_freq').innerText = clickedFreq;
            drawSpectrum();
            alert(`Частота ${clickedFreq} MHz установлена!`);
```

```
        }
      };
    }

    initTheme();
    initCharts();
    setupSpectrumClick();
    drawSpectrum();
    drawCompassRose();

    document.addEventListener('DOMContentLoaded', function() {
      load();
      loadHistory();
      setInterval(load, 5000);
      setInterval(loadHistory, 60000);
    });
</script>
</body>
</html>
```